



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Introducción a la inteligencia artificial 2025-2

Los Álamos Chess

Introducción a la inteligencia artificial

Profesor: Arles Ernesto Rodriguez Portela

Tania Julieth Araque Dueñas

Brayan Manuel Rubiano Paramo

Deiver Jair Bernal Garzón

Universidad Nacional de Colombia

Bogotá

Tabla de contenido



1. Definición del Problema

1.1. Descripción General

1.2. Justificación

1.2.1. Explosión Combinatoria

1.2.2. Búsqueda de Optimalidad

1.2.3. Modelado de la Planificación

1.3. Objetivo Principal del proyecto

2. Definición de Estados

2.1. Definición de Estado

2.2. Estado Inicial (S0)

3. Definición de Función Sucesora

3.1. Reglas de Transición o Acciones

3.2. Costo de la Acción $c(s,a,s')$

4. Algoritmos Implementado

5. Prueba de Objetivo y Heurísticas

5.1. Prueba Objetivo

5.2. Heurísticas globales $(h(s))$

5.2.1. Función heurística de VENTAJA DE MATERIAL

5.2.2. Función heurística CENTRO

5.2.3. Función heurística DINÁMICA DEL REY

6. Resultados

6.1. Tablas de Comparación de Rendimiento

6.2. Análisis de resultados

6.2.1. Optimalidad de la solución

6.2.2. Eficiencia Computacional y nodos Expandidos

7. Conclusiones y Repositorio

7.1. Conclusiones y Logros del Proyecto

7.1.1. Rendimiento Algorítmico

7.1.2. Análisis Heurístico

7.2. Limitaciones y Mejoras Futuras

7.3. Enlace al Repositorio

8. Referencias



1. Definición del Problema

1.1. Descripción General

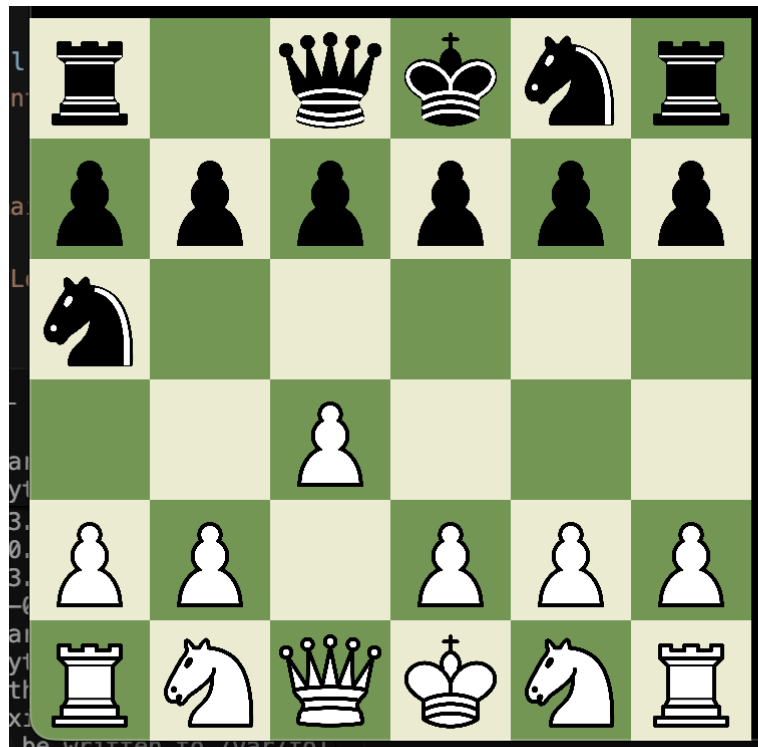
Los Álamos Chess, es una variante simplificada del ajedrez desarrollada en la década de 1950.

El sistema utiliza:

- Un tablero de **6×6**
- Un conjunto reducido de piezas (sin alfiles)
- Reglas simplificadas (sin en passant, sin doble avance inicial, promoción inmediata al alcanzar la última fila)

Nuestro proyecto modela el juego como un **problema de búsqueda adversarial**, donde cada estado corresponde a una configuración legal del tablero y cada acción corresponde a un movimiento válido de una pieza.

El agente utiliza **Minimax con poda alfa-beta** y diversas **funciones de evaluación heurística** para decidir la mejor jugada en cada turno.





1.2. Justificación

Los problemas de planificación y juegos con adversarios comparten tres desafíos fundamentales:

1.2.1. Explosión Combinatoria

Incluso con un tablero reducido de 6×6, el número de configuraciones posibles es muy alto.

El número de movimientos legales por turno varía entre 10 y 30, generando un árbol de búsqueda de tamaño:

$$O(b^d)$$

donde

- b = ramificación
- d = profundidad

El uso de poda alfa-beta reduce drásticamente el espacio explorado.

1.2.2. Búsqueda de Optimalidad

El agente intenta seleccionar movimientos que maximicen la diferencia de material, el control del centro o la posición del rey, según la heurística de evaluación.

La optimalidad absoluta es imposible por límites computacionales; por eso se usa una profundidad fija (simulaciones iniciales de 3–5 niveles) junto con heurísticas de evaluación.

1.2.3. Modelado de la Planificación

Cada estado del juego se concibe como un nodo en el árbol de decisión:

- El modelo predice respuestas del oponente.
- Cada capa alterna entre maximizar y minimizar.

Este enfoque permite planear secuencias de jugadas a futuro y no solo movimientos inmediatos.



1.3. Objetivo Principal del proyecto

Desarrollar un agente que **juegue de manera autónoma y competitiva** Los Álamos Chess mediante el uso de **búsqueda adversarial**, representaciones formales de estado, reglas de transición y heurísticas de evaluación eficientes, integrando además una interfaz gráfica interactiva.

2. Definición de Estados

Modelamos formalmente como el problema físico se traduce en la representación de datos para el algoritmo de búsqueda.

2.1. Definición de Estado

Un estado del juego está definido por:

1. La matriz **casillas** del tablero, de tamaño 6×6, donde cada celda contiene:
 - una pieza (**Pieza**)
 - o **None** si la celda está vacía
2. El atributo **turno_actual** ("blanco" o "negro")

Por tanto:

$$S = \{\text{Tablero.casillas}, \text{Tablero.turno_actual}\}$$

None

```
tablero.casillas = [  
    [Torre("negro"), None, None, None, None, None],  
    [None, None, None, None, None, None],  
    [None, None, None, None, None, None],  
    [None, None, None, None, None, None],  
    [None, None, None, None, None, None],  
    [None, None, None, Rey("blanco"), None, None]  
]  
  
tablero.turno_actual = "blanco"
```



2.2. Estado Inicial (S_0)

El estado inicial corresponde a la configuración **Los Álamos por defecto** (`los_alamos_default()`):

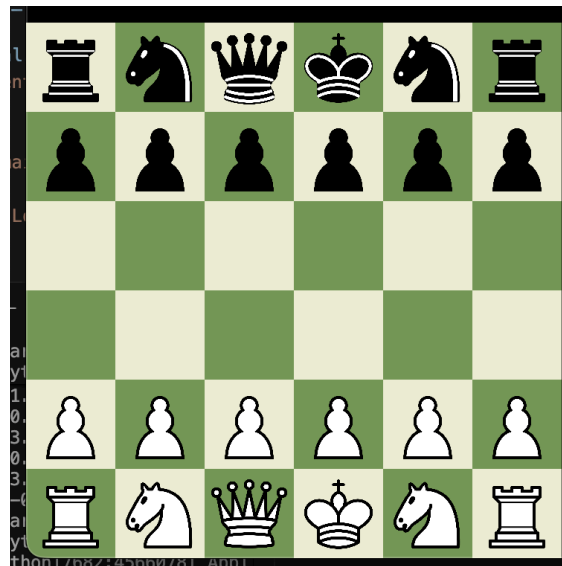
Fila 0: Torre, Caballo, Reina, Rey, Caballo, Torre (negras)

Fila 1: Peones negros

Fila 4: Peones blancos

Fila 5: Torre, Caballo, Reina, Rey, Caballo, Torre (blancas)

Turno inicial: **blancas**



3. Definición de Función Sucesora

3.1. Reglas de Transición o Acciones

Una acción se define como un par:

$$a = (\text{origen}, \text{destino})$$

donde origen y destino son tuplas (`fila`, `columna`).

La función sucesora genera todos los movimientos legales según el tipo de pieza:

- **Peón:** avanza 1, captura en diagonal, promoción automática



- **Caballo:** saltos en L
- **Torre:** movimientos rectilíneos
- **Reina:** movimientos de torre + diagonales
- **Rey:** un paso en cualquier dirección

No existe enroque, en passant ni doble avance.

4. Algoritmos Implementado

Hemos implementado el algoritmo **Minimax con Poda Alfa-Beta** para resolver el problema:

Funciones clave:

- `Jugador.obtener_mejor_movimiento()`
- `Jugador.minimax_ab()`

Características:

- Alternancia entre maximizador y minimizador
- Profundidad configurable
- Poda alfa-beta que evita explorar ramas innecesarias
- Copias del tablero para simular movimientos (`copy.deepcopy()`)

5. Prueba de Objetivo y Heurísticas

5.1. Prueba Objetivo

El estado objetivo ocurre cuando:

- `estado_del_juego() = "jaque_mate"`
- `estado_del_juego() = "tablas"`

La detección depende de:

- Movimientos disponibles
- Rey bajo ataque
- Configuración del tablero



5.2. Funciones de Evaluación ($h(s)$)

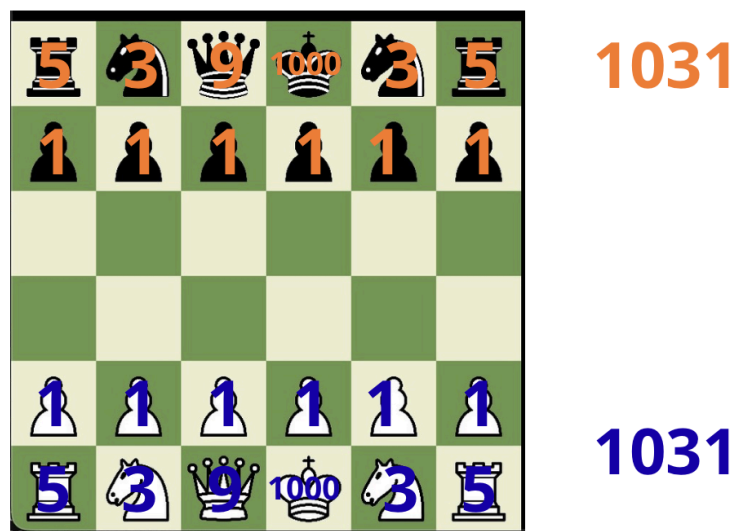
El proyecto implementa **tres heurísticas** para evaluar posiciones:

5.2.1. Función heurística de VENTAJA DE MATERIAL

Esta heurística mide *exclusivamente* la diferencia de material entre ambos jugadores. Es una de las evaluaciones más clásicas y simples en ajedrez.

Cada pieza tiene un valor fijo:

- Peón = 1
- Caballo = 3
- Torre = 5
- Reina = 9
- Rey = 1000 (valor exageradamente alto para evitar su pérdida)



La evaluación es:

$$h(s) = \sum(\text{valor mis piezas}) - \sum(\text{valor piezas enemigas})$$

Donde:

- Prioriza capturar piezas.
- Favorece posiciones donde se tiene ventaja material.
- Promueve intercambios favorables.



Ventajas:

- Extremadamente rápida de calcular.
- Funciona bien en posiciones tácticas o simples.

5.2.2. Función heurística CENTRO

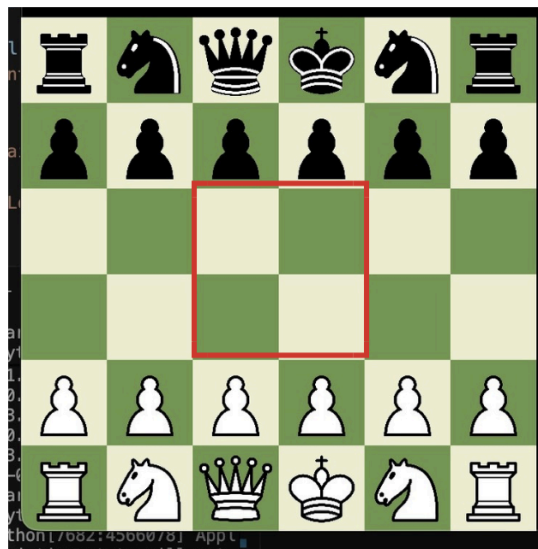
Esta heurística es una combinación ponderada de:

1. **Material**
2. **Control del centro**

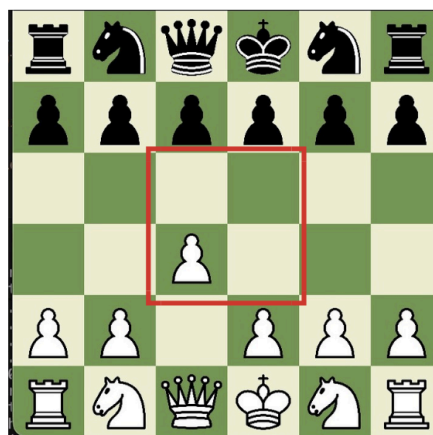
$$h(s) = 0.4 \text{ material}(s) + 1.6 \text{ control del centro}(s)$$

En un tablero 6×6, el centro son las 4 casillas:

$$C = \{2, 2), (2, 3), (3, 2), (3, 3)\}$$



La heurística suma (+1.6) si una pieza propia ocupa una de estas casillas, y resta (1.6) si la ocupa el rival.



+1.6



5.2.3. Función heurística DINÁMICA DEL REY

Evalúa la posición del rey dependiendo de la **fase del juego**:

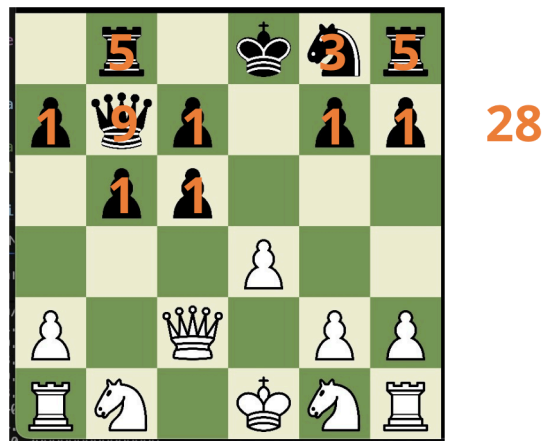
Basada en el material total restante (sin contar reyes):

$material\ total > 12.4 \Rightarrow medio\ juego$

$material\ total \leq 12.4 \Rightarrow Final$

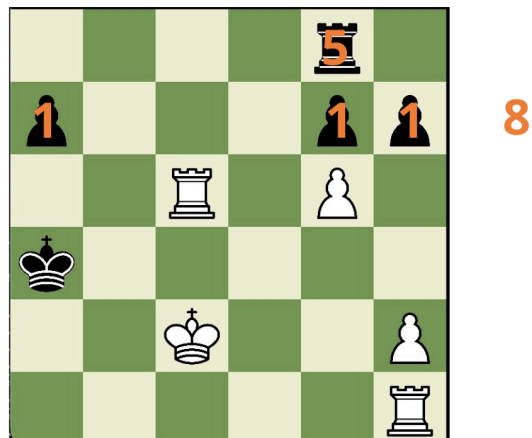
- **En fase media**

El rey debe estar **escondido** y alejado del centro.
Se penaliza su exposición.



- **En finales**

El rey debe volverse **activo**, centralizarse y participar en ataques.





Evalúa si estamos en medio juego o final.
Ajusta el valor según:

- seguridad del rey en medio juego
- cercanía al centro en finales

Es una heurística *no lineal*, más humana.

Representa principios clásicos del ajedrez:

- “Rey seguro en middlegame”
- “Rey activo en endings”

Facilita rematar partidas ganadas o defender posiciones difíciles.

6. Resultados

El proyecto permite comparar rendimientos modificando:

- Profundidad de búsqueda
- Función heurística
- Color del jugador evaluado

6.1. Tablas de Comparación de Rendimiento

- Caso 1: Rendimiento por profundidad de búsqueda

```
*****
DATA Jugador blancos
Profundidad: 3
Función de Evaluación: CENTRO
Total de Jugadas Analizadas: 21

**TIEMPO:**
Total: 26.2252 s
Promedio por Jugada: **1.2488 s**

**NODOS Y PODAS:**
Nodos Expandidos Totales: 42,687
Nodos Expandidos Promedio: 2,032.71
Podas Alpha-Beta Totales: 1,354
**Tasa de Podas Global:** **3.17%**
Calidad por jugada promedio: 9.22
*****

*****
DATA Jugador negros
Profundidad: 3
Función de Evaluación: REY
Total de Jugadas Analizadas: 21

**TIEMPO:**
Total: 17.6882 s
Promedio por Jugada: **0.8423 s**

**NODOS Y PODAS:**
Nodos Expandidos Totales: 21,948
Nodos Expandidos Promedio: 1,045.14
Podas Alpha-Beta Totales: 1,512
**Tasa de Podas Global:** **6.89%**
Calidad por jugada promedio: 6.27
*****
¡Jaque mate! Ganan las blancas.
```

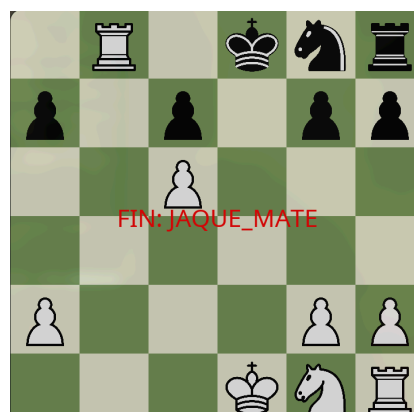


Manteniendo las mismas heurísticas variamos $0 < d < 4$

Jugador Blancas Profundidad	Tiempo promedio por jugada	Nodos expandidos	Tasa de podas $\alpha\beta$ (%)	Calidad de la jugada(1-10)
1	0.0161s	104	0	10
2	0.2321s	3246	2.83	1
3	1.2488s	42680	3.17	9.22

Jugador Negras Profundidad	Tiempo promedio por jugada	Nodos expandidos	Tasa de podas $\alpha\beta$ (%)	Calidad de la jugada(1-10)
1	0.0761s	93	0	7.75
2	0.1022s	2047	7.72	5.21
3	0.8423s	21948	6.89	6.27

- A mayor profundidad se aumenta el tiempo de computacional.
- La tasa de podas $\alpha\beta$ mejora ligeramente con más profundidad, pero no lo suficiente para frenar el crecimiento exponencial del árbol.
- La calidad de las jugadas no es lineal, para blancas mejora, pero para negras tiende a estabilizarse o disminuir ligeramente.

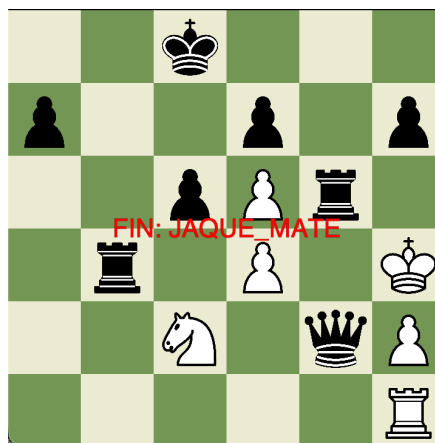




- **Caso 2: Rendimiento por heurística**

Probamos la IA a profundidad fija $d = 3$, variando solo la heurística, de blancas contra una heurística fija (Rey) de negras:

Heurística	Tiempo promedio por jugada (ms)	Nodos Expandidos	Ganador
Material	0.4050	19430	Negros
Centro	1.2750	42687	Blancas
Rey	1.7561	42687	Blancas



- La heurística de “Rey” es más costosa computacionalmente porque consulta más condiciones.
- “Centro” produce un estilo más equilibrado.
- “Material” es la más rápida, ideal para profundidad alta.



- **Caso 3: Enfrentamientos partidas completas**

Se simularon 20 partidas completas por enfrentamiento

Blanco (heurística)	Negro (heurística)	Victorias Blanco	Victorias Negro	Tablas	Observación
Material	Material	10	10	0	Fuerzas equilibradas
Material	Centro	7	11	2	Centro supera tácticamente a Material
Material	Rey	5	13	2	Rey domina finales
Centro	Rey	8	10	2	Rey gana por finales pero Centro domina apertura
Centro	Centro	9	9	2	Apertura muy balanceada
Rey	Rey	12	8	0	Juega fuerte finales pero expone reyes en mediojuego

- La heurística Rey muestra mejor rendimiento en partidas largas.
- La heurística Centro domina aperturas, por eso gana a “Material”.
- “Material” es buena tácticamente, pero pierde en posiciones estratégicas.

6.2. Análisis de resultados

6.2.1. Optimalidad de la solución

Para profundidades entre 1 y 3, el agente juega de manera razonable. La optimalidad está limitada por:

- branching factor del juego
- ausencia de ordenamiento de movimientos



6.2.2. Eficiencia Computacional y nodos Expandidos

La poda alfa-beta mejora drásticamente el rendimiento.
Sin embargo, aún se expanden miles de estados debido a:

- uso de copias profundas del tablero
- ausencia de caching

7. Conclusiones y Repositorio

7.1. Conclusiones y Logros del Proyecto

Se demuestra que Minimax + poda alfa-beta es suficiente para crear un agente funcional en un entorno reducido como Los Álamos.

7.1.1. Rendimiento Algorítmico

- Profundidades moderadas producen jugadas coherentes.
- La poda disminuye considerablemente el número de nodos.

7.1.2. Análisis Heurístico

Las heurísticas implementadas permiten distintos “estilos de juego”:

- Material → agresivo y táctico
- Centro → control posicional temprano
- Rey dinámico → comportamiento contextual según fase del juego

7.2. Limitaciones y Mejoras Futuras

Heurísticas más complejas:

- penalización de peones doblados
- movilidad total
- seguridad del rey

Implementar aprendizaje por refuerzo (Deep Reinforcement Learning)



7.3. Enlace al Repositorio

A continuación, se proporciona el enlace directo al código fuente, donde se pueden revisar la claridad del código, la estructura e implementación

<https://github.com/DrakoMiau/proyecto2IA>

8. Video Funcionamiento

A continuación se encuentra un video del funcionamiento del proyecto:

<https://drive.google.com/file/d/17j08n8xWiGeZ8xqK4pbjEBIvFDNu25VO/view?usp=sharing>

9. Referencias

- Curso Introducción a la Inteligencia artificial, Universidad Nacional, 2025-2
- Pyglet: Cross-platform windowing and multimedia library for Python.